

PlugSure CSMS — server deployment

Nine steps from a bare Ubuntu VM to an EV charge point holding a live WebSocket. Every step states what correct output looks like, so you can confirm each one before moving on.

DOCUMENT	AUDIENCE	PLATFORM	CONTAINS
Deployment runbook	Installing engineer	Ubuntu 24.04 · Docker	No credentials

PLACEHOLDERS IN THIS DOCUMENT

Every value shown as `example.com` , `<ANGLE_BRACKETS>` or an obvious dummy string is a placeholder. **All credentials, the real hostname and the database passwords are supplied separately** and appear nowhere in this document.

Three settings that are not optional

READ BEFORE CONFIGURING ANYTHING

OCPP_MIN_SECURITY_PROFILE=2 — the software's own default is `0` , which means *no per-charger authentication at all*. At `0` , any host that can reach the OCPP port can open a session impersonating any charge point and write into billing data. The API key issued to a hardware vendor does not restrict this, because it is a different port and a different mechanism.

OCPP_TRUST_PROXY_PROTO=true — TLS terminates at Caddy, and without this the application cannot tell that it did, so it refuses every charger with `403` . The gateway refuses to *start* in that combination, so a misconfiguration surfaces immediately rather than during a site visit.

The application must run as the `plugsure_app` database role, never as the database owner. A superuser bypasses every row-level security policy. The process refuses to start that way under `NODE_ENV=production` . The migration step creates this role for you — do not create it by hand.

Prerequisites

ITEM	SPECIFICATION	WHY
VM	Ubuntu 24.04 LTS, 2 vCPU / 4 GB RAM, 20 GB disk	PostgreSQL and both application processes on one host. The protocol frame log is verbose — plan for disk growth.
Static IP	Elastic IP or equivalent reserved address	The address is written into charge point firmware. If it moves, every unit needs a physical site visit.
DNS	A hostname you control, e.g. <code>ocpp.example.com</code>	Let's Encrypt requires a real hostname. Charge points reject self-signed certificates.
Inbound ports	<code>443</code> and <code>80</code> from anywhere; <code>22</code> from the admin network only	443 for chargers, 80 for the ACME challenge, 22 for administration. Do not open 9200 or 5432.
Outbound	HTTPS to Docker Hub, the Debian/Ubuntu repositories and Let's Encrypt	The container image is built on the VM. Confirm egress before starting — a blocked registry stops the build at step 6.

FIX THE HOSTNAME BEFORE ANY HARDWARE IS CONFIGURED

The Central System URL is written into every charge point's firmware. Changing it afterwards means visiting each unit. Confirm the hostname with the project owner before commissioning.

Step-by-step deployment

Run these in order. Each step depends on the previous one having succeeded.

01 Install packages

ON THE VM

```
$ sudo apt-get update
$ sudo apt-get install -y ca-certificates curl postgresql-client-16 \
    docker.io docker-compose-v2 debian-keyring debian-archive-keyring apt-transport-https

# Caddy, from its own repository
$ curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/gpg.key \
    | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
$ curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt \
    | sudo tee /etc/apt/sources.list.d/caddy-stable.list
$ sudo apt-get update && sudo apt-get install -y caddy
```

CONFIRM

```
$ docker compose version → Docker Compose version v2.x
$ caddy version           → v2.x
```

02 Point DNS at the VM and verify it

One A record, TTL 300:

DNS ZONE

NAME	TYPE	VALUE	TTL
ocpp.example.com.	A	<RESERVED_IP>	300

VERIFY BEFORE REQUESTING A CERTIFICATE

```
$ dig +short ocpp.example.com
$ curl -sI http://ocpp.example.com/ | head -1
```

EXPECTED

`dig` returns the reserved IP and nothing else. If it returns an old address or nothing, wait for propagation before continuing.

CERTIFICATE AUTHORITY RATE LIMIT

Let's Encrypt permits five failed validations per hostname per hour. Discovering a firewall problem against the production endpoint locks you out for an hour. Confirm DNS resolves and port 80 reaches the VM first.

03 Place the application on the VM

ON THE VM

```
$ sudo mkdir -p /opt/plugsure && sudo chown $USER /opt/plugsure
$ cd /opt/plugsure && unzip ~/plugsure.zip && cd plugsure
$ npm ci --dry-run
```

EXPECTED

added N packages ...

An `ETARGET` error means an out-of-date copy of the source. Request a current archive rather than working around it.

04 Create the environment file

Generate four secrets. **Record them in the project password manager before continuing.** Changing `AUDIT_HMAC_KEY` later invalidates every audit record already written.

GENERATE

```
$ openssl rand -hex 32 # AUDIT_HMAC_KEY
$ openssl rand -hex 32 # SECRETS_KEY
$ openssl rand -hex 24 # POSTGRES_PASSWORD
$ openssl rand -hex 24 # POSTGRES_APP_PASSWORD
```

/OPT/PLUGSURE/PLUGSURE/.ENV

```
NODE_ENV=production
TZ=Asia/Jakarta

# Database. The application connects as the SECOND role, not the first.
POSTGRES_PASSWORD=<GENERATED>
POSTGRES_APP_PASSWORD=<GENERATED>

# No defaults. Startup fails without these.
AUDIT_HMAC_KEY=<GENERATED>
SECRETS_KEY=<GENERATED>

# The three mandatory settings from page 1.
OCPM_MIN_SECURITY_PROFILE=2
OCPM_TRUST_PROXY_PROTO=true
OCPM_AUTO_ADOPT=false

# Caddy is on the same host and is the only proxy in front.
API_TRUSTED_PROXIES=127.0.0.1

# Bind application ports to loopback. Caddy reaches them; the internet does not.
OCPM_BIND=127.0.0.1
API_BIND=127.0.0.1
```

RESTRICT PERMISSIONS

```
$ chmod 600 .env
```

WHY TWO DATABASE PASSWORDS

Schema migrations require DDL privileges and run as the owner. The gateway and API run as `pluginsure_app`, which cannot alter the schema and cannot bypass row-level security. The migration step provisions that role from `POSTGRES_APP_PASSWORD`.

05 Configure Caddy and obtain the certificate

The supplied `Caddyfile` ships with `example.id` placeholders. Substitute the hostname and contact address you were given:

ON THE VM

```
$ sudo cp deploy/Caddyfile /etc/caddy/Caddyfile

# replace BOTH placeholders with the real values supplied to you
$ sudo sed -i 's/ocpp\.example\.id/<OCPP_HOSTNAME>/g' /etc/caddy/Caddyfile
$ sudo sed -i 's/ops@example\.id/<OPS_CONTACT_EMAIL>/' /etc/caddy/Caddyfile
$ grep -n 'example\.id' /etc/caddy/Caddyfile # must return nothing
$ sudo caddy validate --config /etc/caddy/Caddyfile
$ sudo systemctl reload caddy
$ sudo journalctl -u caddy -f
```

WATCH FOR

certificate obtained successfully

The supplied Caddyfile already handles the two details that matter: it proxies `/ocpp/*` to `127.0.0.1:9220` without altering the `Sec-WebSocket-Protocol` header, and it *assigns* `X-Forwarded-Proto` rather than appending to a client-supplied value, so a charge point cannot claim a secure transport it is not using. Do not edit those directives.

REHEARSE WITH THE STAGING ENDPOINT

Uncomment the `acme_ca` line in the Caddyfile global block, reload, confirm issuance works end to end, then comment it out **and delete the staging certificate**, or Caddy continues serving it:

```
$ sudo rm -rf /var/lib/caddy/.local/share/caddy/certificates
$ sudo systemctl restart caddy
```

06 Start the stack

ON THE VM

```
$ docker compose build
$ docker compose up -d
$ docker compose ps
```

EXPECTED

migrate	Exited (0)
postgres	healthy
gateway	healthy
api	healthy

Start order is enforced by the compose file: `postgres` healthy → `migrate` exits 0 → `gateway` and `api`. The applications cannot start against an unmigrated database.

Now verify the two conditions that fail silently.

DATABASE ROLE

```
$ docker compose logs api | grep 'row-level security'
```

MUST REPORT

row-level security is in force for application queries

Any other result means the application is running as a superuser and tenant isolation has one layer of protection instead of two. Stop and correct the database URL.

GATEWAY SECURITY

```
$ docker compose logs gateway | grep -E 'listening|profile'
```

MUST SHOW

```
"minSecurityProfile": 2, "tls": "expected at proxy"
```

If the gateway exited instead, the error names which of TLS or `OCPP_TRUST_PROXY_PROTO` is missing. That refusal is intentional: the alternative is a gateway that appears healthy while rejecting every charge point.

07 Initialise data and capture the API key

ON THE VM

```
$ docker compose run --rm migrate node dist/db/seed.js
```

EXPECTED – RECORD THIS LINE

```
API key (shown once):  psk_<prefix>_<secret>
```

The key is displayed once and cannot be retrieved afterwards. Store it in the project password manager immediately and hand it to the project owner.

KEY HANDLING

This key carries full organisation permissions. Any key issued from it can only ever carry a subset of its own permissions, so a delegated key cannot escalate. Issue narrower keys for other parties rather than sharing this one.

The operator console is an internal surface. Reach it over an SSH tunnel rather than exposing port 9200:

FROM AN ADMINISTRATOR WORKSTATION

```
$ ssh -N -L 9200:127.0.0.1:9200 ubuntu@<VM_ADDRESS>
# then browse to http://127.0.0.1:9200/
```

08 Register a charge point — four calls, in this order

A production gateway refuses unknown identities and requires credentials, so both must exist *before* the hardware connects. The order matters: setting the security profile before a key exists is refused by design.

THROUGH THE SSH TUNNEL

```
$ export K=<API_KEY>
$ export API=http://127.0.0.1:9200
$ export ID=<CHARGE_POINT_IDENTITY> # exactly as the vendor configures it

# 1. register the identity – created in pending_adoption
$ curl -sS -X POST $API/v1/charge-points \
  -H "Authorization: Bearer $K" -H 'Content-Type: application/json' \
  -d '{"ocppIdentity\\":\\"$ID\\",\\"siteId\\":\\"<SITE_UUID>\\"}'

# 2. issue the AuthorizationKey – RETURNED ONCE
$ curl -sS -X POST $API/v1/charge-points/$ID/authorization-key \
  -H "Authorization: Bearer $K"

# 3. set the security profile (PUT, not POST)
$ curl -sS -X PUT $API/v1/charge-points/$ID/security-profile \
  -H "Authorization: Bearer $K" -H 'Content-Type: application/json' \
  -d '{"profile":2}'

# 4. activate – this is what permits transactions
$ curl -sS -X POST $API/v1/charge-points/$ID/activate \
  -H "Authorization: Bearer $K"
```

EXPECTED

```
1 → {"ok":true,"status":"pending_adoption","next":[...]}
2 → {"key":"<RETURNED ONCE>","warning":"This key is shown once..."}
3 → {"ok":true,"profile":2}
4 → {"ok":true}
```

Obtain `siteId` from `GET /v1/charge-points`, or create a site in the console first. The response to step 1 lists the remaining steps.

CONFIGURE THE CHARGE POINT BEFORE RAISING THE PROFILE

Set the `AuthorizationKey` on the hardware first, *then* perform step 3. The reverse order leaves a unit that cannot connect and an engineer on site with no way to proceed.

09 Validate with the supplied simulator before hardware arrives

A virtual charge point ships with the application. Point it at the live gateway and run a complete session, so that a real unit is not the first thing to connect.

ON THE VM, OR ANY HOST WITH NODE 22

```
$ export PLUGSURE_KEY=<API_KEY>
$ npx tsx tools/simulator/autel-sim.ts \
  --url wss://ocpp.example.com/ocpp \
  --id <CHARGE_POINT_IDENTITY> --auth-key <AUTHORIZATION_KEY> \
  --scenario happy
```

EXPECTED

```
--- happy: PASS ---
```

The scenario checks, against the API, that the system recorded exactly the energy the simulator delivered.

The following scenarios are also worth running and cost nothing: `offline-replay` (store-and-forward after a network drop), `reconnect-storm`, and `meter-rollover`.

Diagnosing a charge point that will not connect

Every connection attempt is recorded before the WebSocket upgrade completes — source address, requested path, offered subprotocols, transport security, whether credentials were presented, and the outcome. This log is the fastest route to a diagnosis.

READ THE ATTEMPT LOG

```
$ curl -sS "$API/v1/connection-attempts?limit=20" -H "Authorization: Bearer $K" | jq
```

STATUS	MEANING	RESOLUTION
101	Connected successfully	—
400	No supported subprotocol offered	Set the unit to OCPP 1.6J. Only <code>ocpp1.6</code> is advertised, by design.
401	Missing or incorrect credentials	The HTTP Basic username must be the charge point identity byte-for-byte; the password is the AuthorizationKey.
403	Insecure transport at security profile 2	The unit is using <code>ws://</code> and must use <code>wss://</code> . If every unit reports this, check <code>OCPP_TRUST_PROXY_PROTO</code> .
404	Identity not registered	Perform step 8.1. The identity is case-sensitive and character-exact.
Pending	Connected and authenticated, not yet activated	Not a fault. Perform step 8.4.

Routine operations

TASK	COMMAND
Follow gateway logs	<code>docker compose logs -f gateway</code>
Warnings and errors only, last 30 min	<code>docker compose logs --since 30m api jq -c 'select(.level >= 40)'</code>

TASK	COMMAND
Caddy / TLS logs	<code>sudo journalctl -u caddy -f</code>
API health	<code>curl -s http://127.0.0.1:9200/healthz jq</code>
Gateway health and live charger count	<code>curl -s http://127.0.0.1:9220/healthz jq</code>
Restart one service	<code>docker compose restart gateway</code>
Database backup	<code>pg_dump "\$DATABASE_URL" -Fc -f /var/backups/plugsure-\$(date +%F-%H%M).dump</code>
Apply an updated build	<code>docker compose build && docker compose up -d</code>

TWO OPERATIONAL NOTES

Take a database backup before every update. There is no automated backup schedule in this deployment; add one to your own monitoring if the service is contracted to you.

Deploy the API and the gateway independently where possible. They run as separate containers from the same image on purpose, so that updating one does not drop the WebSocket connections held by the other.

Escalation

When reporting a problem, include **the charge point identity** and **a UTC timestamp**. Incidents are looked up by those two fields, and the frame log can then be replayed message by message in both directions.

PlugSure CSMS — deployment runbook · contains no credentials · all secrets and the production hostname supplied separately